

C++ Pointers & Arrays — Consolidated Lab Manual

Data Structures / Programming Fundamentals Lab

This lab brings together the fundamentals of pointers and arrays covered so far into one hands-on session. Each topic below gives brief key points, a syntax + basic example to get comfortable, and two intermediate programs that apply the concept to a small problem.

1. Double Pointers

- A double pointer (int **pp) is a pointer that stores the address of another pointer, not the address of a plain variable.
- Declared as: ***data_type **pointerName;***
- Common uses: modifying a pointer's value from inside a function, dynamic 2D arrays, arrays of pointers.
- *pp gives back the inner pointer; **pp dereferences twice to reach the actual value.

Syntax & Basic Example

```
int a = 5;
int *p = &a;
int **pp = &p;

cout << **pp;    // prints 5 – two levels of dereferencing
```

```
#include <iostream>
using namespace std;

int main() {
    int a = 5;
    int *p = &a;
    int **pp = &p;

    cout << "Value of a: " << a << endl;
    cout << "Value via p (*p): " << *p << endl;
    cout << "Value via pp (**pp): " << **pp << endl;

    cout << "Address of a: " << &a << endl;
    cout << "Value of p (address of a): " << p << endl;
    cout << "Value of pp (address of p): " << pp << endl;

    return 0;
}
```

Program 1: Swap Two Pointers

A function needs a double pointer parameter whenever it must change what the caller's pointer itself points to — here, swapping which variable each pointer refers to.

```
#include <iostream>
using namespace std;

void swapPointers(int **p1, int **p2) {
    int *temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}

int main() {
    int a = 10, b = 20;
    int *p1 = &a;
    int *p2 = &b;

    cout << "Before swap: *p1 = " << *p1 << ", *p2 = " << *p2 << endl;
    swapPointers(&p1, &p2);
    cout << "After swap: *p1 = " << *p1 << ", *p2 = " << *p2 << endl;

    return 0;
}
```

Program 2: Allocate Memory Through a Function

The function must return a newly allocated address back to main() through the caller's pointer, so it needs a double pointer parameter.

```
#include <iostream>
using namespace std;

void allocateAndFill(int **ptr, int value) {
    *ptr = new int;    // allocate memory
    **ptr = value;     // store the value
}

int main() {
    int *p = nullptr;
    allocateAndFill(&p, 99);

    cout << "Value allocated: " << *p << endl;

    delete p;
    return 0;
}
```

2. 1D Array: Indexes and Pointers

- An array can be traversed with index syntax (`arr[i]`) or with pointer arithmetic (`*(ptr + i)`).
- The array name decays to a pointer to its first element — `int *ptr = arr;` is valid with no `&` needed.
- A pointer can also be incremented (`ptr++`) inside a loop to move to the next element directly.

Syntax & Basic Example

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;

    cout << "Using index:" << endl;
    for (int i = 0; i < 5; i++)
        cout << arr[i] << " ";
    cout << endl;

    cout << "Using pointer:" << endl;
    for (int i = 0; i < 5; i++)
        cout << *(ptr + i) << " ";
    cout << endl;

    return 0;
}
```

Program 1: Sum & Average Using Pointer Traversal

```
#include <iostream>
using namespace std;

int main() {
    int arr[6] = {12, 7, 19, 3, 25, 10};
    int *ptr = arr;
    int sum = 0;

    for (int i = 0; i < 6; i++) {
        sum += *ptr;
        ptr++;           // move to next element
    }

    double avg = (double)sum / 6;
    cout << "Sum = " << sum << endl;
    cout << "Average = " << avg << endl;

    return 0;
}
```

Program 2: Reverse an Array (Two-Pointer Technique)

```
#include <iostream>
using namespace std;

int main() {
    int arr[7] = {1, 2, 3, 4, 5, 6, 7};
    int *start = arr;
    int *end = arr + 6;

    while (start < end) {
        int temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }

    for (int i = 0; i < 7; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

3. 2D Array: Indexes and Pointers (Both Methods)

- A 2D array can be accessed with indices (`arr[i][j]`), with a row pointer (`int (*ptr)[cols]`), or with a flattened plain pointer (`*(ptr + i*cols + j)`).
- A row pointer's step size is one whole row; a flattened pointer's step size is one element, so the row width must be multiplied in manually.

Syntax & Basic Example (All Three Access Styles)

```
#include <iostream>
using namespace std;

int main() {
    int arr[2][3] = { {1,2,3}, {4,5,6} };

    cout << "Using indices:" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++)
            cout << arr[i][j] << " ";
        cout << endl;
    }

    cout << "Using row pointer:" << endl;
    int (*rowPtr)[3] = arr;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++)
            cout << (*(rowPtr + i) + j) << " ";
    }
}
```

```

        cout << endl;
    }

    cout << "Using flattened pointer:" << endl;
    int *flatPtr = &arr[0][0];
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++)
            cout << *(flatPtr + i*3 + j) << " ";
        cout << endl;
    }

    return 0;
}

```

Program 1: Transpose a Matrix

```

#include <iostream>
using namespace std;

int main() {
    int arr[2][3] = { {1,2,3}, {4,5,6} };
    int transposed[3][2];

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            transposed[j][i] = arr[i][j];

    cout << "Transposed matrix:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++)
            cout << transposed[i][j] << " ";
        cout << endl;
    }

    return 0;
}

```

Program 2: Row & Column Sums Using a Row Pointer

```

#include <iostream>
using namespace std;

int main() {
    int arr[3][3] = { {1,2,3}, {4,5,6}, {7,8,9} };
    int (*ptr)[3] = arr;

    cout << "Row sums:" << endl;
    for (int i = 0; i < 3; i++) {
        int rowSum = 0;
        for (int j = 0; j < 3; j++)
            rowSum += (*(ptr+i)+j);
        cout << "Row " << i << ": " << rowSum << endl;
    }

    cout << "Column sums:" << endl;
}

```

```

    for (int j = 0; j < 3; j++) {
        int colSum = 0;
        for (int i = 0; i < 3; i++)
            colSum += (*(ptr+i)+j);
        cout << "Column " << j << ": " << colSum << endl;
    }

    return 0;
}

```

4. Passing a 1D Array to a Function

- An array decays to a pointer when passed, so the function parameter can be written as `int arr[], int arr[size]`, or `int *arr` — all mean the same thing.
- Since size information is lost, the size must always be passed as a separate parameter.
- Because only the pointer is passed (not a copy), the function can modify the caller's original array.

Syntax & Basic Example

```

#include <iostream>
using namespace std;

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int arr[5] = {1,2,3,4,5};
    printArray(arr, 5);
    return 0;
}

```

Program 1: Find the Maximum Element

```

#include <iostream>
using namespace std;

int findMax(int *arr, int size) {
    int maxVal = arr[0];
    for (int i = 1; i < size; i++)
        if (arr[i] > maxVal)
            maxVal = arr[i];
    return maxVal;
}

int main() {
    int arr[6] = {4, 9, 2, 7, 5, 1};
    cout << "Max element: " << findMax(arr, 6) << endl;
}

```

```
    return 0;
}
```

Program 2: Reverse the Array In Place

```
#include <iostream>
using namespace std;

void reverseArray(int *arr, int size) {
    int *start = arr;
    int *end = arr + size - 1;
    while (start < end) {
        int temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}

int main() {
    int arr[5] = {1,2,3,4,5};
    reverseArray(arr, 5);

    for (int i = 0; i < 5; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

5. Passing a 2D Array to a Function

- Recap: three signatures work — `func(int arr[row][col])` needs both dimensions as const globals, `func(int arr[][col], int row)` needs only col as a const global, and `func(int *arr, int row, int col)` needs neither but requires manual offset math.

Syntax & Basic Example (Row-Pointer Method)

```
#include <iostream>
using namespace std;

const int COL = 3;

void printUsingGlobalCol(int arr[][COL], int row) {
    for (int i = 0; i < row; i++) {
        for (int j = 0; j < COL; j++)
            cout << arr[i][j] << " ";
        cout << endl;
    }
}
```

```
int main() {
    int arr[2][COL] = { {1,2,3}, {4,5,6} };
    printUsingGlobalCol(arr, 2);
    return 0;
}
```

Program 1: Sum of All Elements (Flattened Pointer Method)

```
#include <iostream>
using namespace std;

int sumMatrix(int *arr, int row, int col) {
    int sum = 0;
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            sum += *((arr + i*col) + j);
    return sum;
}

int main() {
    int arr[2][3] = { {1,2,3}, {4,5,6} };
    cout << "Sum = " << sumMatrix((int*)arr, 2, 3) << endl;
    return 0;
}
```

Program 2: Maximum Element (Row-Pointer Method)

```
#include <iostream>
using namespace std;

const int COL2 = 3;

int findMaxMatrix(int arr[][COL2], int row) {
    int maxVal = arr[0][0];
    for (int i = 0; i < row; i++)
        for (int j = 0; j < COL2; j++)
            if (arr[i][j] > maxVal)
                maxVal = arr[i][j];
    return maxVal;
}

int main() {
    int arr[3][COL2] = { {1,9,3}, {4,5,6}, {7,8,2} };
    cout << "Max = " << findMaxMatrix(arr, 3) << endl;
    return 0;
}
```

6. Dynamic 1D Array: Creation and Passing to a Function

- A dynamic array is created with `new type[size]` and released with `delete[]` once it's no longer needed.
- A dynamic array is passed to a function exactly like a normal array — as a pointer plus a size parameter.
- To change the pointer itself inside a function (e.g. to resize it), the function needs a double pointer parameter.

Syntax & Basic Example

```
#include <iostream>
using namespace std;

int main() {
    int size = 5;
    int *arr = new int[size];

    for (int i = 0; i < size; i++)
        arr[i] = (i+1) * 10;

    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;

    delete[] arr;
    return 0;
}
```

Program 1: Sum of a Dynamic Array

```
#include <iostream>
using namespace std;

int sumArray(int *arr, int size) {
    int sum = 0;
    for (int i = 0; i < size; i++)
        sum += arr[i];
    return sum;
}

int main() {
    int size = 5;
    int *arr = new int[size]{1,2,3,4,5};

    cout << "Sum = " << sumArray(arr, size) << endl;

    delete[] arr;
    return 0;
}
```

Program 2: Resize a Dynamic Array

Resizing requires reassigning the caller's pointer to a brand-new block, so this function needs a double pointer (int **arr).

```
#include <iostream>
using namespace std;

void resizeArray(int **arr, int oldSize, int newSize) {
    int *newArr = new int[newSize];

    for (int i = 0; i < oldSize; i++)
        newArr[i] = (*arr)[i];
    for (int i = oldSize; i < newSize; i++)
        newArr[i] = 0;

    delete[] *arr;
    *arr = newArr;
}

int main() {
    int *arr = new int[3]{1,2,3};

    resizeArray(&arr, 3, 6);

    for (int i = 0; i < 6; i++)
        cout << arr[i] << " ";
    cout << endl;

    delete[] arr;
    return 0;
}
```

7. Dynamic 2D Array: Creation and Passing to a Function

- Single-pointer (flattened) method: one new int[rows*cols] block; access with *(arr + i*cols + j); needs only one delete[].
- Double-pointer (array of pointers) method: new int*[rows] followed by a separate new int[cols] per row; allows jagged rows; needs one delete[] per row plus one for the row-pointer array.

Syntax & Basic Example (Double-Pointer Method)

```
#include <iostream>
using namespace std;

int main() {
    int rows = 2, cols = 3;

    int **arr = new int*[rows];
    for (int i = 0; i < rows; i++)
        arr[i] = new int[cols];
}
```

```

int val = 1;
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        arr[i][j] = val++;

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++)
        cout << arr[i][j] << " ";
    cout << endl;
}

for (int i = 0; i < rows; i++)
    delete[] arr[i];
delete[] arr;

return 0;
}

```

Program 1: Matrix Addition (Flattened Method)

```

#include <iostream>
using namespace std;

void addMatrices(int *a, int *b, int *result, int rows, int cols) {
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            *(result + i*cols + j) = *(a + i*cols + j) + *(b + i*cols + j);
}

int main() {
    int rows = 2, cols = 2;
    int *a = new int[rows*cols]{1,2,3,4};
    int *b = new int[rows*cols]{5,6,7,8};
    int *result = new int[rows*cols];

    addMatrices(a, b, result, rows, cols);

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++)
            cout << *(result + i*cols + j) << " ";
        cout << endl;
    }

    delete[] a; delete[] b; delete[] result;
    return 0;
}

```

Program 2: Sum a Jagged 2D Array (Double-Pointer Method)

Since rows may differ in length, a colCounts[] array records each row's length and is passed alongside the double pointer.

```
#include <iostream>
using namespace std;

int sumJaggedArray(int **arr, int rows, int *colCounts) {
    int sum = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < colCounts[i]; j++)
            sum += arr[i][j];
    return sum;
}

int main() {
    int rows = 3;
    int colCounts[3] = {2, 4, 3};

    int **arr = new int*[rows];
    for (int i = 0; i < rows; i++)
        arr[i] = new int[colCounts[i]];

    int val = 1;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < colCounts[i]; j++)
            arr[i][j] = val++;

    cout << "Sum = " << sumJaggedArray(arr, rows, colCounts) << endl;

    for (int i = 0; i < rows; i++)
        delete[] arr[i];
    delete[] arr;

    return 0;
}
```

8. Returning Pointers from Functions

- Never return the address of a local (stack) variable — it becomes a dangling pointer as soon as the function returns.
- It is safe to return a pointer to memory allocated with new, since it stays alive until the caller deletes it.

Program 1: Return a Pointer to a Single Dynamic Value

```
#include <iostream>
using namespace std;

int* createInt(int value) {
    int *p = new int(value);
    return p;
}
```

```
int main() {
    int *ptr = createInt(42);
    cout << "Value = " << *ptr << endl;

    delete ptr;
    return 0;
}
```

Program 2: Return a Pointer to a Dynamically Created Array

```
#include <iostream>
using namespace std;

int* createArray(int size) {
    int *arr = new int[size];
    for (int i = 0; i < size; i++)
        arr[i] = (i+1) * (i+1);    // squares

    return arr;
}

int main() {
    int size = 5;
    int *arr = createArray(size);

    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;

    delete[] arr;
    return 0;
}
```